Team Notebook

Last Dance

March 3, 2026

Contents

1 convex-hull-trick **2**
- 1.1 cht-deque . . . 2
- 1.2 line-container . . . 2

2 data-structures **2**
- 2.1 centroid-decomposition . . . 2
- 2.2 dsu-rollback . . . 3
- 2.3 dsu-sum . . . 3
- 2.4 fast-rmq . . . 3
- 2.5 fenwick . . . 3
- 2.6 hld . . . 4
- 2.7 kd-tree . . . 4
- 2.8 lca . . . 4
- 2.9 lichao-tree . . . 5
- 2.10 mo-algorithm . . . 5
- 2.11 mos3D . . . 6
- 2.12 offline-connectivity . . . 6
- 2.13 persistent-segment-tree . . . 7
- 2.14 policy-based-data-structures . . . 7
- 2.15 sack-on-tree . . . 7
- 2.16 segment-tree-2d . . . 8
- 2.17 segment-tree . . . 8
- 2.18 virtual-tree . . . 9

3 geometry **9**
- 3.1 angle . . . 9
- 3.2 check-intersect . . . 9
- 3.3 convex-hull . . . 9
- 3.4 find-intersect . . . 9
- 3.5 line . . . 10
- 3.6 point . . . 10
- 3.7 polygon-area . . . 11

4 graph **11**
- 4.1 2sat . . . 11
- 4.2 articulation-point . . . 11
- 4.3 bridge . . . 12
- 4.4 flow . . . 12
- 4.5 hopcroft-karp . . . 13
- 4.6 min-cost-max-flow . . . 13
- 4.7 minimum-vertex-cover . . . 15
- 4.8 spfa . . . 15

5 math **15**
- 5.1 chinese-remainer . . . 15
- 5.2 count-prime . . . 15
- 5.3 eulerphi . . . 16
- 5.4 extended-euclid . . . 16
- 5.5 factorization . . . 16
- 5.6 fft . . . 17
- 5.7 fraction . . . 18
- 5.8 gauss . . . 18
- 5.9 ntt . . . 19
- 5.10 primitive-root . . . 19
- 5.11 sieve-range . . . 20
- 5.12 xor-transform . . . 20

6 misc **20**
- 6.1 compare-function . . . 20
- 6.2 hash-un-weighted-tree . . . 20
- 6.3 int128 . . . 20
- 6.4 main . . . 21
- 6.5 misc . . . 21
- 6.6 random . . . 21
- 6.7 stresstest . . . 21

7 string **21**
- 7.1 aho-corasick . . . 21
- 7.2 duval-prefixes . . . 22
- 7.3 duval . . . 22
- 7.4 hash61 . . . 22
- 7.5 hashing-2d . . . 23
- 7.6 kmp . . . 23
- 7.7 manacher . . . 23
- 7.8 suffix-array . . . 24
- 7.9 z-function . . . 25

1 convex-hull-trick

1.1 cht-deque

```
using ll = long long;
const ll inf = 1.1e18;
// y = a * x + b
struct Line {
    ll a, b;
    ll calc(ll x) { return a * x + b; }
};
ll upper_divide(ll a, ll b) {
    return a / b + ((a ^ b) > 0 && a % b);
}

// Complexity: O(N+Q)
// To know usage of this code, see [line container]
// Require increasing add/query (and manual deque)
// "add_dec" and "query_dec" using for decreasing add/query
struct CHT_Deque {
    deque<pair<ll, Line>> lines;
    void add(ll a, ll b) {
        Line l = {a, b};
        if(lines.empty()) {
            lines.emplace_back(-inf, l);
        }
        else {
            Line l_back = lines.back().second;
            assert(l.a >= l_back.a);

            if(l.a == l_back.a) {
                if(l.b <= l_back.b) return;
                lines.pop_back();
            }
            while(!lines.empty()) {
                l_back = lines.back().second;
                ll x = lines.back().first;
                ll new_x = upper_divide(l_back.b - l.b, l.a -
                    l_back.a);
                if(new_x <= x) lines.pop_back();
                else break;
            }
            if(lines.empty()) {
                lines.emplace_back(-inf, l);
            }
            else {
                ll new_x = upper_divide(l_back.b - l.b, l.a -
                    l_back.a);
                lines.emplace_back(new_x, l);
            }
        }
    }
    ll query(ll x) {
        while(lines.size() > 1 && next(lines.begin())->first
            <= x) {
            lines.pop_front();
            lines.front().first = -inf;
        }
        return lines.front().second.calc(x);
    }
    void add_dec(ll a, ll b) {
        add(-a, b);
    }
    ll query_dec(ll x) {
        return query(-x);
    }
};
```

1.2 line-container

```
/**
 * Author: Simon Lindholm
 * Date: 2017-04-20
 * License: CC0
 * Source: own work
 * Description: Container where you can add lines of the
     form kx+m, and query maximum values at points x.
 *  Useful for dynamic programming (``convex hull trick'').
 * Time: O(\log N)
 * Status: stress-tested
 */
#pragma once
#include <bits/stdc++.h>
#define ll long long
using namespace std;

struct Line {
 mutable ll k, m, p;
 bool operator<(const Line& o) const { return k < o.k; }
 bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
 // (for doubles, use inf = 1/.0, div(a,b) = a/b)
 static const ll inf = LLONG_MAX;
 ll div(ll a, ll b) { // floored division
  return a / b - ((a ^ b) < 0 && a % b); }
 bool isect(iterator x, iterator y) {
  if (y == end()) return x->p = inf, 0;
  if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
  else x->p = div(y->m - x->m, x->k - y->k);
  return x->p >= y->p;
 }
 void add(ll k, ll m) {
  auto z = insert({k, m, 0}), y = z++, x = y;
  while (isect(y, z)) z = erase(z);
  if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
  while ((y = x) != begin() && (--x)->p >= y->p)
   isect(x, erase(y));
 }
 ll query(ll x) {
  assert(!empty());
  auto l = *lower_bound(x);
  return l.k * x + l.m;
 }
};
```

2 data-structures

2.1 centroid-decomposition

```
// path(u, v) = path'(u, w) + path'(w, v) with w is lca of (
    u, v) in centroid tree
const int N = 1e5 + 5;
vector<pair<int, int>> adj[N];
int del[N], sz[N];
int dfs_sz(int u, int p) {
    sz[u] = 1;
    for (auto [v, w] : adj[u]) {
        if (v != p && !del[v]) sz[u] += dfs_sz(v, u);
    }
    return sz[u];
}
int centroid(int u, int p, int n) {
    for (auto [v, w] : adj[u]) {
        if (v != p && !del[v] && sz[v] > n / 2) return
            centroid(v, u, n);
    }
    return u;
}
void solve(int u, int p) {
    int n = dfs_sz(u, p);
    int c = centroid(u, p, n);
    del[c] = 1;
    for (auto [v, w] : adj[c]) {
        if (del[v]) continue;
        // solve subtree of centroid here
```

```
    }
    for (auto [v, w] : adj[c]) {
        if (!del[v]) solve(v, c);
    }
}
```

2.2 dsu-rollback

```
struct DSUrb {
    vector<int> par, sz;
    stack<vector<int>> s;
    int cc;
    void init(int n) {
        cc = n; par.resize(n);
        sz.resize(n, 1);
        iota(par.begin(), par.end(), 0);
    }
    int get(int x) {
        if (x == par[x]) return x;
        return get(par[x]);
    }
    bool join(int u, int v) {
        u = get(u); v = get(v);
        if (u == v) return false;
        if (sz[u] < sz[v]) swap(u, v);
        s.push({ u, v, sz[v] });
        par[v] = u; sz[u] += sz[v];
        return true;
    }
    void rollback(int cnt) {
        while (cnt--) {
            auto a = s.top(); s.pop();
            par[a[1]] = a[1];
            sz[a[0]] -= a[2];
            sz[a[1]] = a[2];
        }
    }
    int same(int a, int b){
        return get(a) == get(b);
    }
};
```

2.3 dsu-sum

```
// luu gia tri tri tu u toi root, gia tri rieng cua root dai
     dien cho ca cay xai chung
```

2.4 fast-rmq

```
// RMQ with sparse table O<N, 1>
template<typename T>
struct RMQ {
    static const int b = 30;
    vector<T> dat;
    vector<int> mask, a;
    function<bool(T, T)> g;
    int n;
    RMQ() : n(0) {}
    int f(int a, int b) {
        return g(dat[a], dat[b]) ? a : b;
    }
    RMQ(vector<T>& dat, function<bool(T, T)> g) {
        this->g = g;
        this->dat = dat;
        n = dat.size();
        mask = vector<int>(n);
        a = vector<int>(n);
        int cur_mask = 0;
        for (int i = 0; i < n; i++) {
            cur_mask = (cur_mask << 1) & ((1 << b) - 1);
            while (cur_mask > 0 && f(i, i - msb_index(lsb(
                 cur_mask))) == i) {
                cur_mask ^= lsb(cur_mask);
            }
            cur_mask |= 1;
            mask[i] = cur_mask;
        }
        for (int i = 0; i < n / b; i++) {
            a[i] = small(b * i + b - 1);
        }
        for (int j = 1; (1 << j) <= n / b; j++) for (int i =
             0; i + (1 << j) <= n / b; i++) {
            a[n / b * j + i] = f(a[n / b * (j - 1) + i], a[n
                 / b * (j - 1) + i + (1 << (j - 1))]);
        }
    }
    int lsb(int x) {
        return x & -x;
    }
    int msb_index(int x) {
        return __builtin_clz(1) - __builtin_clz(x);
    }
    int small(int r, int size = b) {
        int dist_from_r = msb_index(mask[r] & ((1 << size) -
             1));
        return r - dist_from_r;
    }
    T query(int l, int r) {
        assert(l <= r);
        if (r - l + 1 <= b) return dat[small(r, r - l + 1)];
        int ans = f(small(l + b - 1), small(r));
        int x = l / b + 1, y = r / b - 1;
        if (x <= y) {
            int j = msb_index(y - x + 1);
            ans = f(ans, f(a[n / b * j + x], a[n / b * j + y
                 - (1 << j) + 1]));
        }
        return dat[ans];
    }
};
```

2.5 fenwick

```
// for multi-dimension D, vector<pair<int, int>> = {x1 - 1,
     x2} {y1 - 1, y2} ...
// if sum (0-1) % 2 = D % 2 -> + else - , note : sum (0-1)
     is choose p1 - 1 or p2
// 1 - indexed
struct fenwick {
 int n;
 vector<int> bit;
 fenwick() {}
 fenwick(int n) {
  this->n = n + 5;
  bit.resize(n + 5);
 }
 void add(int pos, int val) {
  while (pos <= n) {
   bit[pos] += val;
   pos += pos & -pos;
  }
 }
 int get(int pos) {
  int res = 0;
  while (pos > 0) {
   res += bit[pos];
   pos -= pos & -pos;
  }
  return res;
 }
 int get(int l, int r) {
  return get(r) - get(l - 1);
 }
 // add x : pos -> n : x += x & -x
 // get x : pos -> 0 : x -= x & -x
 int find(int k) {
```

```
  int sum = 0, pos = 0;
  for (int i = __lg(n); i >= 0; i--) {
   if (pos + (1 << i) < n && sum + bit[pos + (1 << i)] < k)
        {
    sum += bit[pos + (1 << i)];
    pos += (1 << i);
   }
  }
  return pos + 1;
 }
};
```

2.6 hld

```
// subtree v [in[v], out[v] - 1]
int n;
vector<vector<int>> adj;
vector<int> sz(n), in(n), out(n), head(n), par(n), h(n);
function<void(int, int)> dfs_sz = [&](int u, int p) {
 sz[u] = 1;
 for (auto& v : adj[u]) {
  if (v == p) continue;
  dfs_sz(v, u);
  sz[u] += sz[v];
  if (sz[v] > sz[adj[u][0]]) {
   swap(v, adj[u][0]);
  }
 }
};
dfs_sz(0, -1);
int t = 1;
function<void(int, int)> dfs_hld = [&](int u, int p) {
 in[u] = t++;
 for (auto v : adj[u]) {
  if (v == p) continue;
  par[v] = u;
  h[v] = h[u] + 1;
  head[v] = (v == adj[u][0] ? head[u] : v);
  dfs_hld(v, u);
 }
 out[u] = t;
};
dfs_hld(0, -1);
function<int(int, int)> query = [&](int u, int v){
 int ans = 0;
 for (; head[u] != head[v]; v = par[head[v]]){
  if (h[head[u]] > h[head[v]]) swap(u, v);
  //ckmax(ans, get(in[head[v]], in[v]));
 }
 if (h[u] > h[v]) swap(u, v);
 //ckmax(ans, get(in[u], in[v]));
 return ans;
};
```

2.7 kd-tree

```
#include <bits/stdc++.h>
using namespace std;
const int inf = 1e9;
// KD-Tree
struct Node {
    Node* l = nullptr; Node* r = nullptr;
    int xl = inf; int xr = -inf;
    int yl = inf; int yr = -inf;
    long long val = 0;
};
void combine(Node* t) {
    t->val = max(t->l->val, t->r->val);
}
// d (dimension) : 0 -> x, 1 -> y
vector<pair<int, int>> b;
Node* build(int l, int r, int d = 0) {
    Node* t = new Node();
    for (int i = l; i <= r; i++) {
        auto [x, y] = b[i];
        t->xl = min(t->xl, x); t->xr = max(t->xr, x);
        t->yl = min(t->yl, y); t->yr = max(t->yr, y);
    }
    if (l == r) {
        return t;
    }
    int n = r - l + 1;
    nth_element(b.begin() + l, b.begin() + l + n / 2, b.begin
         () + r + 1, [&](auto i, auto j) {
        return d ? i.second < j.second : i.first < j.first;
    });
    t->l = build(l, l + n / 2 - 1, d ^ 1);
    t->r = build(l + n / 2, r, d ^ 1);
    combine(t);
    return t;
}
long long query(Node* t, int xl, int xr, int yl, int yr) {
    if (!t->val) return 0;
    if (t->xr < xl || t->xl > xr || t->yr < yl || t->yl > yr
         || !t->val) return 0;
    if (t->xl >= xl && t->xr <= xr && t->yl >= yl && t->yr <=
         yr) return t->val;
    return max(query(t->l, xl, xr, yl, yr), query(t->r, xl,
         xr, yl, yr));
}
void update(Node* t, int x, int y, long long val) {
    if (t->xl > x || t->xr < x || t->yl > y || t->yr < y)
         return;
    if (x == t->xl && x == t->xr && y == t->yl && y == t->yr)
         {
        t->val = max(t->val, val);
        return;
    }
    update(t->l, x, y, val); update(t->r, x, y, val);
    combine(t);
}
const int N = 2e5;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, dx, dy;
    cin >> n >> dx >> dy;
    vector<tuple<int, int, int, int>> a(n);
    for (auto &[h, x, y, p] : a) cin >> x >> y >> h >> p;
    sort(a.begin(), a.end());
    for (auto [h, x, y, p] : a) b.emplace_back(x, y);
    sort(b.begin(), b.end());
    auto root = build(0, b.size() - 1);
    for (auto [h, x, y, p] : a) {
        int xl = max(1, x - dx);
        int yl = max(1, y - dy);
        long long k = query(root, xl, min(N, x + dx), yl, min
             (N, y + dy));
        if (k + p > 0) update(root, x, y, k + p);
    }
    cout << query(root, 1, N, 1, N);
    return 0;
}
```

2.8 lca

```
template <class T>
struct RMQ { // 0-based
  vector<vector<T>> rmq;
  T kInf = numeric_limits<T>::max();
  void build(const vector<T>& V) {
    int n = V.size(), on = 1, dep = 1;
    while (on < n) on *= 2, ++dep;
    rmq.assign(dep, V);

    for (int i = 0; i < dep - 1; ++i)
```

```
      for (int j = 0; j < n; ++j) {
        rmq[i + 1][j] = min(rmq[i][j], rmq[i][min(n - 1, j +
             (1 << i))]);
      }
  }
  T query(int a, int b) { // [a, b)
    if (b <= a) return kInf;
    int dep = 31 - __builtin_clz(b - a); // log(b - a)
    return min(rmq[dep][a], rmq[dep][b - (1 << dep)]);
  }
};

struct LCA { // 0-based
  vector<int> enter, depth, exxit;
  vector<vector<int>> G;
  vector<pair<int, int>> linear;
  RMQ<pair<int, int>> rmq;
  int timer = 0;
  LCA() {}
  LCA(int n) : enter(n, -1), exxit(n, -1), depth(n), G(n),
       linear(2 * n) {}
  void dfs(int node, int dep) {
    linear[timer] = {dep, node};
    enter[node] = timer++; depth[node] = dep;
    for (auto vec : G[node])
    if (enter[vec] == -1) {
      dfs(vec, dep + 1);
      linear[timer++] = {dep, node};
    }
    exxit[node] = timer;
  }
  void add_edge(int a, int b) {
    G[a].push_back(b); G[b].push_back(a);
  }
  void build(int root) {
    dfs(root, 0);
    rmq.build(linear);
  }
  int query(int a, int b) {
    a = enter[a], b = enter[b];
    return rmq.query(min(a, b), max(a, b) + 1).second;
  }
  int dis(int a, int b) {
    return depth[a] + depth[b] - 2 * depth[query(a, b)];
  }
};
```

2.9 lichao-tree

```
struct Line {
    int a, b;
    int get(int x) {
        return a * x + b;
    }
};
struct Node {
    int l, r; //child id
    int s, e; //range
    Line line;
};
struct Li_Chao {
    vector<Node> tree;
    const int inf = 2e18; // double of max_value possible
    // inf if Li_Chao_min
    // -inf if Li_Chao max
    void init(int s, int e) {
        tree.push_back({ -1, -1, s, e, { 0, inf } });
    }
    void update(int node, Line v) {
        int s = tree[node].s, e = tree[node].e;
        int m = (s + e) >> 1;
        Line low = tree[node].line, high = v;
        if (low.get(s) < high.get(s)) swap(low, high); //
             change sign
        if (low.get(e) >= high.get(e)) { // change sign
            tree[node].line = high; return;
        }
        if (low.get(m) > high.get(m)) { // change sign
            tree[node].line = high;
            if (tree[node].r == -1) {
                tree[node].r = tree.size();
                tree.push_back({ -1, -1, m + 1, e, { 0, inf }
                     });
            }
            update(tree[node].r, low);
        }
        else {
            tree[node].line = low;
            if (tree[node].l == -1) {
                tree[node].l = tree.size();
                tree.push_back({ -1, -1, s, m, { 0, inf } });
            }
            update(tree[node].l, high);
        }
    }
    void update(Line v){
        update(0, v);
    }
    int query(int node, int x) {
        if (node == -1) return inf;
        int s = tree[node].s, e = tree[node].e;
        int m = (s + e) >> 1;
        if (x <= m) return min(tree[node].line.get(x), query(
             tree[node].l, x));
        else return min(tree[node].line.get(x), query(tree[
             node].r, x));
    }
    int query(int x){
        return query(0, x);
    }
};
// seg.init(-2e9, 2e9); // min_range, max_range x in queries
```

2.10 mo-algorithm

```
/*
Let in[u] <= in[v], p = lca(u, v)
Case 1 : p = u -> [in[u], in[v]]
Case 2 : p != u -> [out[u], in[v]] [in[p], in[p]]
*/
class Query {
public:
 int id, l, r;
 Query(int id, int l, int r) :
  id(id), l(l), r(r), prior(ConvHilbert(l, r)) {}
 bool operator< (const Query& i) const { return prior < i.
      prior; }
private:
 long long prior;
 static long long ConvHilbert(int x, int y) {
  long long ret = 0;
  for (int i = 17, cx, cy; ~i; i--) {
   cx = x >> i & 1, cy = y >> i & 1;
   ret = ret << 2 | cx << 1 | cx ^ cy;
   cy || (swap(x, y), cx) && (x = ~x, y = ~y);
  }
  return ret;
 }
};
bool cmp(pair<int, int> p, pair<int, int> q) {
    if (p.first / BLOCK_SIZE != q.first / BLOCK_SIZE)
        return p < q;
    return (p.first / BLOCK_SIZE & 1) ? (p.second < q.second)
          : (p.second > q.second);
}
```

2.11 mos3D

```
const int maxN = 1e5 + 5;
struct query{
    int l, r, k, t, id;
};
// Luu lai thong tin cap nhat: Sau khi thuc hien truy van
     nay, a[index] = old se tro thanh a[index] = next. Luu
     lai nhu vay nham muc dich thuan tien khi dao nguoc truy
      van.
struct update{
    int index, old, next;
};
vector<update> update_queries;
vector<query> queries;
int n, q, a[maxN], b[maxN], has[maxN] = {}, S, l = 1, r = 0,
      t = -1;
int occ[maxN] = {}, ans[maxN], ord[maxN] = {}, f[maxN] = {};
pair<int, int> range[maxN] = {};
bool cmp(const query &a, const query &b){
    if(a.l / S != b.l / S)
        return a.l < b.l;
    else if(a.r / S != b.r / S){
        if((a.l / S) % 2 == 1)
            return a.r < b.r;
        else
            return a.r > b.r;
    }
    return a.t < b.t;
}
void add(int x) {
}
void rmv(int x) {
}
void run_update(int i, int x){
    // Neu i nam trong [l, r], viec cap nhat se anh huong den
          dap an
    if(l <= i && i <= r){
        rmv(a[i]);
        add(x);
    }
    a[i] = x;
}
signed main(){
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0)
         ;
    memset(ans, -1, sizeof ans);
    S = pow(2LL * n * n, 1. / 3.) + 1;
    for(int i = 1; i <= q; i++){
        int t;
        cin >> t;
        int u, v;
        cin >> u >> v;
        if(t == 2){
            update_queries.push_back({u, b[u], v});
            b[u] = v;
        }else{
         int k;
         cin >> k;
            queries.push_back({u, v, k, (int)update_queries.
                 size() - 1, i});
            has[i] = 1;
        }
    }
    sort(queries.begin(), queries.end(), cmp);
    for(query &q : queries){
        // Truy van cap nhat
        while(t < q.t) t++, run_update(update_queries[t].
             index, update_queries[t].next);
        // Dao nguoc truy van cap nhat
        while(t > q.t) run_update(update_queries[t].index,
             update_queries[t].old), t--;
        while (l > q.l) add(a[--l]);
        while (r < q.r) add(a[++r]);
        while (l < q.l) rmv(a[l++]);
        while (r > q.r) rmv(a[r--]);
        // answer query here
    }
}
```

2.12 offline-connectivity

```
pair<int, int> q[MAXN];
int cnt = 1, nq;
map<pair<int, int>, int> mp;
vector<vector<int>> e;
vector<vector<pair<int, int>>> st(1 << 19);
void update(int u, int v, int f, int s, int id = 1, int l =
     1, int r = 1 << 18) {
    if (r < u || l > v) return;
    if (l >= u && r <= v) {
        st[id].emplace_back(f, s);
        return;
    }
    int mid = (l + r) >> 1;
    update(u, v, f, s, id << 1, l, mid);
    update(u, v, f, s, id << 1 | 1, mid + 1, r);
}
DSUrb s;
void divide(int id = 1, int l = 1, int r = 1 << 18) {
    int z = 0;
    for (auto x : st[id]) {
        z += (int)s.join(x.first, x.second);
    }
    if (l == r) {
        if (l < cnt) {
            cout << (s.same(q[l].first, q[l].second) ? "YES"
                 : "NO") << '\n';
        }
        s.rollback(z);
        return;
    }
    int mid = (l + r) >> 1;
    divide(id << 1, l, mid); divide(id << 1 | 1, mid + 1, r);
    s.rollback(z);
}
void solve() {
    vector<int> state(m);
    for (int& i : state) cin >> i;
    for (int i = 0; i < m; i++) {
        if (edge[i].first > edge[i].second) swap(edge[i].
             first, edge[i].second);
        if (state[i]) {
            mp[edge[i]] = cnt;
        }
    }
    cin >> nq;
    s.init(n);
    for (int i = 0; i < nq; i++) {
        int t, a, b;
        cin >> t >> a;
        a--;
        if (t == 1) {
            if (state[a]) {
                if (mp[edge[a]] != cnt) {
                    e.push_back({ edge[a].first, edge[a].
                         second, mp[edge[a]], cnt - 1 });
                }
                mp.erase(edge[a]);
            }
            else {
                mp[edge[a]] = cnt;
            }
            state[a] ^= 1;
        }
        else {
            cin >> b;
            b--;
            if (a > b) swap(a, b);
```

```
            q[cnt++] = { a,b };
        }
    }
    for (auto x : mp) {
        if (x.second != cnt) {
            e.push_back({ x.first.first, x.first.second, x.
                second, cnt - 1 });
        }
    }
    for (int i = 0; i < e.size(); i++) update(e[i][2], e[i
        ][3], e[i][0], e[i][1]);
    divide();
}
```

2.13 persistent-segment-tree

```
#define N 1000000 // N must double of update or add
#define LG 20 // log
int L[N * LG], R[N * LG], val[N * LG], root[N];
long long sum[N * LG];
int M = 0, p = 2, n = 200001;
void update(int id, int l, int r, int x) {
    val[id]++; sum[id] += x;
    if (l != r) {
        int mid = (l + r) >> 1;
        int u = L[id], v = R[id];
        if (x <= mid) {
            L[id] = p++;
            tie(L[L[id]], R[L[id]], val[L[id]], sum[L[id]]) =
                {L[u], R[u], val[u], sum[u]};
            update(L[id], l, mid, x);
        }
        else {
            R[id] = p++;
            tie(L[R[id]], R[R[id]], val[R[id]], sum[R[id]]) =
                {L[v], R[v], val[v], sum[v]};
            update(R[id], mid + 1, r, x);
        }
    }
}
void create_node(int x) {
    root[++M] = p++;
    tie(L[root[M]], R[root[M]], val[root[M]], sum[root[M]]) =
        {L[root[M - 1]], R[root[M - 1]], val[root[M - 1]],
        sum[root[M - 1]]};
    update(root[M], 1, n, x);
}
long long get_occ(int v, int u, int x, int y, int l, int r)
    {
    if (r < x || l > y) return 0;
    if (l >= x && r <= y) return val[u] - val[v];
    int mid = (l + r) >> 1;
    return get_occ(L[v], L[u], x, y, l, mid) + get_occ(R[v],
        R[u], x, y, mid + 1, r);
}
long long get_occ(int l, int r, int x, int y) {
    return get_occ(root[l - 1], root[r], x, y, 1, n);
}
long long get_sum(int v, int u, int x, int y, int l, int r)
    {
    if (r < x || l > y) return 0;
    if (l >= x && r <= y) return sum[u] - sum[v];
    int mid = (l + r) >> 1;
    return get_sum(L[v], L[u], x, y, l, mid) + get_sum(R[v],
        R[u], x, y, mid + 1, r);
}
long long get_sum(int l, int r, int x, int y) {
    return get_sum(root[l - 1], root[r], x, y, 1, n);
}
long long get_kth(int v, int u, int x, int l, int r) {
    if (l == r) return r;
    int k = val[L[u]] - val[L[v]];
    int mid = (l + r) >> 1;
    if (k >= x) return get_kth(L[v], L[u], x, l, mid);
    return get_kth(R[v], R[u], x - k, mid + 1, r);
}
long long get_kth(int l, int r, int x) {
    return get_kth(root[l - 1], root[r], x, 1, n);
}
void build(int id, int l, int r) {
    val[id] = sum[id] = 0;
    if (l == r) return;
    int mid = (l + r) >> 1;
    L[id] = p++; R[id] = p++;
    build(L[id], l, mid);
    build(R[id], mid + 1, r);
}

// root[0] = 1
// build(1, 1, n);
// node in order of call create_node
```

2.14 policy-based-data-structures

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
typedef tree<int, null_type, less_equal<int>, rb_tree_tag,
    tree_order_statistics_node_update> ordered_multiset; //
    like multiset
// typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> order_set; // can
    use .erase(x) (x is value)
.insert(x); // insert x
.order_of_key(val + 1) - .order_of_key(val) > 0; // check
    val exists
.erase(.find_by_order(.order_of_key(c))) // erase c , should
    check exists before
.find_by_order(pos) // s[pos] : 0-indexed
.order_of_key(x) // n.o element less than x
```

2.15 sack-on-tree

```
function<int(int, bool)> dfs_solve = [&](int u, bool keep) {
        int mx = -1, big_child = -1, cur = 0;
        for (auto [v, c] : adj[u]) {
            if (sz[v] > mx) {
                mx = sz[v];
                big_child = v;
            }
        }
        for (auto [v, c] : adj[u]) {
            if (v != big_child) dfs_solve(v, false);
        }
        if (big_child != -1) {
            cur = max(cur, dfs_solve(big_child, true));
            if (f[g[u]] != -1) {
                cur = max(cur, f[g[u]] - h[u]);
            }
            for (int j = 0; j < 22; j++) {
                if (f[g[u] ^ (1 << j)] != -1) {
                    cur = max(cur, f[g[u] ^ (1 << j)] - h[u]);
                }
            }
        }
        for (auto [v, c] : adj[u]) {
            if (v == big_child) continue;
            for (int i = in[v]; i < out[v]; i++) {
                int k = g[et[i]] ^ g[u];
                if ((k & (k - 1)) == 0) {
                    cur = max(cur, h[et[i]] - h[u]);
                }
                k ^= g[u];
                if (f[k] != -1) {
                    cur = max(cur, h[et[i]] + f[k] - 2 * h[u])
                        ;
                }
                for (int j = 0; j < 22; j++) {
```

```
                int l = k ^ (1 << j);
                if (f[l] != -1) {
                    cur = max(cur, h[et[i]] + f[l] - 2 * h[
                        u]);
                }
            }
        }
        for (int i = in[v]; i < out[v]; i++) {
            int k = g[et[i]];
            if (f[k] < h[et[i]]) f[k] = h[et[i]];
        }
    }
    ans[u] = cur;
    if (f[g[u]] < h[u]) {
        f[g[u]] = h[u];
    }
    if (!keep) {
        for (int i = in[u]; i < out[u]; i++) {
            f[g[et[i]]] = -1;
        }
        return 0;
    }
    return cur;
};
```

2.16 segment-tree-2d

```
// 1-indexed
template<typename T, typename F>
struct SegmentTree2D {
 vector<vector<T>> tree;
 size_t szx, szy;
 const T ID;
 F f{};
 SegmentTree2D(size_t n, size_t m, T ID) : SegmentTree2D(n,
     m, ID, F{}) {}
 explicit SegmentTree2D(size_t n, size_t m, T ID, const F& f
     ) :
  szx(Log2(n)), szy(Log2(m)), ID(ID), f(f) {
  tree = vector<vector<T>>(szx << 1, vector<T>(szy << 1, ID)
      );
 }
 static size_t Log2(size_t n) {
  n--;
  for (int i = 0; i < 5; i++) n |= n >> (1 << i);
  return n + 1;
 }
 T get_child(int id_par, int y1, int y2, int id, int l, int
     r) {
  if (r < y1 || l > y2) return ID;
  if (l >= y1 && r <= y2)
   return tree[id_par][id];
  int mid = (l + r) >> 1;
  return f(get_child(id_par, y1, y2, id << 1, l, mid),
      get_child(id_par, y1, y2, id << 1 | 1, mid + 1, r));
 }
 T get_child(int id_par, int y1, int y2) {
  return get_child(id_par, y1, y2, 1, 1, szy);
 }
 T get(int x1, int y1, int x2, int y2, int id, int l, int r)
     {
  if (r < x1 || l > x2) return ID;
  if (l >= x1 && r <= x2)
   return get_child(id, y1, y2);
  int mid = (l + r) >> 1;
  return f(get(x1, y1, x2, y2, id << 1, l, mid), get(x1, y1,
      x2, y2, id << 1 | 1, mid + 1, r));
 }
 T get(int x1, int y1, int x2, int y2) {
  return get(x1, y1, x2, y2, 1, 1, szx);
 }
 void update_y(int id_par, int l_par, int r_par, int y, T
     val, int id, int l, int r) {
  if (l == r) {
   if (l_par == r_par)
    tree[id_par][id] = val;
   else
    tree[id_par][id] = f(tree[id_par << 1][id], tree[id_par
        << 1 | 1][id]);
  }
  else {
   int mid = (l + r) >> 1;
   if (y <= mid)
    update_y(id_par, l_par, r_par, y, val, id << 1, l, mid);
   else
    update_y(id_par, l_par, r_par, y, val, id << 1 | 1, mid +
        1, r);
   tree[id_par][id] = f(tree[id_par][id << 1], tree[id_par][
       id << 1 | 1]);
  }
 }
 void update_y(int id_par, int l_par, int r_par, int y, T
     val) {
  update_y(id_par, l_par, r_par, y, val, 1, 1, szy);
 }
 void update(int x, int y, T val, int id, int l, int r) {
  if (l != r) {
   int mid = (l + r) >> 1;
   if (x <= mid)
    update(x, y, val, id << 1, l, mid);
   else
    update(x, y, val, id << 1 | 1, mid + 1, r);
  }
  update_y(id, l, r, y, val);
 }
 void update(int x, int y, T val) {
  update(x, y, val, 1, 1, szx);
 }
};
auto f = [&](int a, int b) -> int { return a + b; };
SegmentTree2D<int, decltype(f)> tree(n, n, 0, f);
```

2.17 segment-tree

```
// 1-indexed
template<typename T, typename F>
struct segtree {
    const size_t sz; const T ID; F f{};
    vector<T> tree;
    segtree(size_t n, T ID) : segtree(n, ID, F{}) {}
    explicit segtree(size_t n, T ID, const F& f) :
        sz(Log2(n)), ID(ID), f(f),
        tree(sz << 1, ID) {}
    static size_t Log2(size_t n) {
        n--;
        for (int i = 0; i < 5; i++) n |= n >> (1 << i);
        return n + 1;
    }
    void update(int i, T val) {
        --i |= sz; tree[i] = val;
        while (i >>= 1) tree[i] = f(tree[i << 1], tree[i << 1
            | 1]);
    }
    T query(int l, int r) const {
        T L = ID, R = ID; --l |= sz, --r |= sz;
        while (l <= r) {
            if (l & 1) L = f(L, tree[l++]);
            if (~r & 1) R = f(tree[r--], R);
            l >>= 1, r >>= 1;
        }
        return f(L, R);
    }
};
auto f = [&](int a, int b) -> int { return a < b ? a : b; };
segtree<int, decltype(f)> tree(n, numeric_limits<int>::max()
    , f);
```

2.18 virtual-tree

```
int k = p.size();
ranges::sort(p, [&](int u, int v) {return in[u] < in[v];});
for (int i = 1; i < k; i++) p.push_back(lca(p[i - 1], p[i]))
    ;
ranges::sort(p, [&](int u, int v) {return in[u] < in[v];});
p.erase(ranges::unique(p).begin(), p.end());
for (int i : p) vadj[i].clear();
for (int i = 1; i < p.size(); i++) vadj[lca(p[i - 1], p[i])
    ].push_back(p[i]);
```

3 geometry

3.1 angle

```
#include "point.h"

// Tra ve cos(AOB)
double angle(Point a, Point o, Point b) {
    int d = dot(a - o, b - o);
    return sqrt(sqr_len(o - a)) * sqrt(sqr_len(o - b)) / d;
}
// Tra ve goc luong giac
double polar_angle(Point p) {
    assert(p != Point(0, 0));
    if(comp(p.x) == 0) {
        return (comp(p.y) > 0 ? PI/2 : -PI/2);
    }
    else {
        double ans = atan((double) p.y / p.x);
        if(comp(p.x) < 0) ans += PI;
        if(ans < 0) ans += PI * 2;
        return ans;
    }
}
```

3.2 check-intersect

```
// (Real number coordinate sometime not trustable)
#include "point.h"
// Kiem tra b co nam giua a va c khong (neu a, b, c da thang
    hang)
bool between(Point a, Point b, Point c) {
    return a <= b && b <= c || c <= b && b <= a;
}
// Kiem tra b co nam giua a va c khong (a, b, c chua chac da
    thang hang)
bool on_segment(Point a, Point b, Point p) {
    return ccw(a, b, p) == 0 && between(a, p, b);
}
// Kiem tra AB va CD co diem chung nam hoan toan ben trong
    khong
bool is_inner_intersect(Point a, Point b, Point c, Point d)
    {
    int c1 = ccw(a, b, c);
    int c2 = ccw(a, b, d);
    int c3 = ccw(c, d, a);
    int c4 = ccw(c, d, b);
    return c1 * c2 < 0 && c3 * c4 < 0;
}
// Kiem tra giao diem cua 2 duong thang, tra ve
// 0 neu khong co diem chung
// 1 neu co diem cat nhau o bien
// 2 neu co doan chung
// -1 neu cat nhau hoan toan ben trong
int count_intersect(Point a, Point b, Point c, Point d) {
    if (a == b) {
        return on_segment(c, d, a);
    }
    if (c == d) {
        return on_segment(a, b, c);
    }
    int c1 = ccw(a, b, c);
    int c2 = ccw(a, b, d);
    int c3 = ccw(c, d, a);
    int c4 = ccw(c, d, b);
    if (c1 * c2 < 0 && c3 * c4 < 0) return -1;
    if (c1 * c2 != 0 && c3 * c4 != 0) return 0;
    int cnt = 0;
    if (c1 == 0 && between(a, c, b)) cnt++;
    if (c2 == 0 && between(a, d, b)) cnt++;
    if (c3 == 0 && between(c, a, d)) cnt++;
    if (c4 == 0 && between(c, b, d)) cnt++;
    if(a == c || a == d) cnt--;
    if(b == c || b == d) cnt--;
    return cnt;
}
```

3.3 convex-hull

```
#include "point.h"
// Bao loi khong tinh diem thang hang, nguoc chieu kim dong
    ho
// Stress test https://judge.yosupo.jp/problem/
    static_convex_hull
void convex_hull(vector<Point>& a) {
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    if(a.size() <= 2) return;
    vector<Point> up;
    vector<Point> down;
    Point p1 = a[0], p2 = a.back();
    up.push_back(p1);
    down.push_back(p1);
    for(int i = 1; i < a.size(); i++) {
        if(i == a.size() - 1 || ccw(p1, a[i], p2) > 0) {
            while (up.size() >= 2 && ccw(up[up.size()-2], up[
                up.size()-1], a[i]) <= 0)
                up.pop_back();
            up.push_back(a[i]);
        }
        if(i == a.size() - 1 || ccw(p1, a[i], p2) < 0) {
            while (down.size() >= 2 && ccw(down[down.size() -
                2], down[down.size()-1], a[i]) >= 0)
                down.pop_back();
            down.push_back(a[i]);
        }
    }
    if(up.size() == a.size()) return;
    a = up;
    for(int i = down.size() - 2; i > 0; i--) {
        a.push_back(down[i]);
    }
}
```

3.4 find-intersect

```
#include "check_intersect.h"
#include "line.h"

// center (x, y), radius r
struct Circle {
    num_t x;
    num_t y;
    num_t r;
    Circle() {}
    Circle(num_t x, num_t y, num_t r) {this->x = x; this->y =
        y; this->r = r;}
};

double det(double a, double b, double c, double d) {
    return a*d - b*c;
```

```
}

// Giao diem 2 duong thang
// Tra ve false neu 2 duong thang song song hoac trung nhay
// Nguoc lai tra ve true va giao diem
bool intersect_line_line(Line m, Line n, Point& res) {
    double zn = det(m.a, m.b, n.a, n.b);
    if (abs(zn) < EPS)
        return false;
    res.x = -det(m.c, m.b, n.c, n.b) / zn;
    res.y = -det(m.a, m.c, n.a, n.c) / zn;
    return true;
}

// Tra ve diem giao nhau cua 2 doan thang AB va CD
// Trong truong hop co doan chung, tra ve doan thang giao
     nhau chung theo thu tu tang dan
vector<Point> intersect_segment_segment(Point a, Point b,
     Point c, Point d) {
    int cnt = count_intersect(a, b, c, d);
    if(cnt == 0) {
        return {};
    }
    else if(cnt == -1) { // co diem chung o giua
        Point ans;
        intersect_line_line(to_line(a, b), to_line(c, d), ans
             );
        return {ans};
    }
    else {
        vector<Point> ans;
        if(on_segment(a, b, c)) {ans.push_back(c);}
        if(on_segment(a, b, d)) {ans.push_back(d);}
        if(on_segment(c, d, a)) {ans.push_back(a);}
        if(on_segment(c, d, b)) {ans.push_back(b);}
        sort(ans.begin(), ans.end());
        ans.erase(unique(ans.begin(), ans.end()), ans.end());
        return ans;
    }
}

// Giao diem duong thang va duong tron
vector<Point> intersect_line_circle(Line l, Circle c1) {
    double dx = c1.x;
    double dy = c1.y;
    vector<Point> ans;

    double a = l.a;
    double b = l.b;
    double c = l.c + l.a * dx + l.b * dy;
    double r = c1.r;
    double x0 = -a*c / (sqr(a) + sqr(b));
    double y0 = -b*c / (sqr(a) + sqr(b));
    if (sqr(c) > sqr(r) * (sqr(a) + sqr(b)) + EPS) {
        return ans;
    }
    else if (abs (sqr(c) - sqr(r) * (sqr(a) + sqr(b))) < EPS)
         {
        ans.push_back({x0 + dx, y0 + dy});
        return ans;
    }
    else {
        double d = sqr(r) - sqr(c) / (sqr(a) + sqr(b));
        double mult = sqrt (d / (sqr(a) + sqr(b)));
        double ax, ay, bx, by;
        ax = x0 + b * mult + dx;
        bx = x0 - b * mult + dx;
        ay = y0 - a * mult + dy;
        by = y0 + a * mult + dy;

        ans.push_back({ax, ay});
        ans.push_back({bx, by});
    }
    return ans;
}

// Giao diem 2 duong tron
vector<Point> intersect_circle_circle(Circle c1, Circle c2)
     {
    double dx = c1.x;
    double dy = c1.y;
    c2.x -= c1.x;
    c2.y -= c1.y;
    c1.x = 0;
    c1.y = 0;

    Line l;
    l.a = -2 * c2.x;
    l.b = -2 * c2.y;
    l.c = sqr(c2.x) + sqr(c2.y) + sqr(c1.r) - sqr(c2.r);
    vector<Point> ans = intersect_line_circle(l, c1);
    for(Point& p : ans) {
        p.x += dx;
        p.y += dy;
    }
    return ans;
}
```

3.5 line

```
#include "point.h"
// Duong thang co phuong trinh ax + by + c = 0
struct Line {
    num_t a, b, c;
    void norm() {
        if(a < 0 || a == 0 && b < 0 || a == 0 && b == 0 && c
             < 0) {
            a = -a;
            b = -b;
            c = -c;
        }
        int g = __gcd(abs((ll)a), __gcd(abs((ll)b), abs((ll)c
             )));
        if(g != 0) {
            a /= g;
            b /= g;
            c /= g;
        }
    }
};
// Tra ve duong thang AB
Line to_line(Point a, Point b) {
    if(a == b) {
        return {0, 0, 0};
    }
    Line res;
    res.a = a.y - b.y;
    res.b = b.x - a.x;
    res.c = - (res.a * a.x + res.b * a.y);
    return res;
}
// Tra ve khoang cach giua diem va duong thang
double point_distance_to_line(Point p, Line l) {
    return abs(l.a * p.x + l.b * p.y + l.c) / sqrt(sqr(l.a) +
         sqr(l.b));
}
```

3.6 point

```
#include<bits/stdc++.h>
using namespace std;
#define sqr(x) ((x) * (x))
using num_t = long long;
// using num_t = double;
using ll = long long;
const num_t EPS = 1e-8;
const double PI = acos(-1);
```

```
inline int comp(num_t x) {
    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}
// Diem co toa do (x, y)
// Hoac vector co toa do (x, y)
struct Point {
    num_t x, y;
    void input() {
        cin >> x >> y;
    }
    Point() {}
    Point(num_t x, num_t y) {this->x = x; this->y = y;}
    void norm() {
        if(x < 0 || x == 0 && y < 0) {
            x = -x;
            y = -y;
        }
        int g = __gcd(llabs((ll)x), llabs((ll)y));
        if(g != 0) {
            x /= g;
            y /= g;
        }
    }
};
bool operator < (Point a, Point b) {
    int cx = comp(a.x - b.x);
    if (cx != 0) return cx < 0;
    return comp(a.y - b.y) < 0;
}
bool operator == (Point a, Point b) { return comp(a.x - b.x)
      == 0 && comp(a.y - b.y) == 0; }
bool operator <= (Point a, Point b) { return (a < b) || (a
     == b); }
bool operator > (Point a, Point b) { return b < a; }
bool operator >= (Point a, Point b) { return (b < a) || (a
     == b); }
bool operator != (Point a, Point b) { return !(a == b); }
Point operator + (Point a, Point b) { return Point(a.x + b.x
     , a.y + b.y); }
Point operator - (Point a, Point b) { return Point(a.x - b.x
     , a.y - b.y); }
Point operator * (Point a, num_t x) { return Point(a.x * x,
     a.y * x); }
Point operator / (Point a, num_t x) { return Point(a.x / x,
     a.y / x); }
// Tra ve phep nhan co huong
// Tra ve phep nhan vo huong
num_t operator * (Point a, Point b) { return a.x * b.y - a.y
      * b.x; }
num_t dot(Point a, Point b) { return a.x * b.x + a.y * b.y;
     }
// Tra ve 1 neu a, b, c nguoc chieu kim dong ho (re trai)
//      -1 neu a, b, c cung chieu kim dong ho (re phai)
//       0 neu a, b, c thang hang
int ccw(Point a, Point b, Point c) {
    return comp((b - a) * (c - b));
}
```

3.7 polygon-area

```
// Tra ve 2 lan dien tich da giac
num_t polygon_area(vector<Point>& p) {
    int n = p.size();
    num_t s = 0;
    for(int i = 0; i < n; i++) {
        s += p[i] * p[(i+1)%n];
    }
    return abs(s);
}
```

4 graph

4.1 2sat

```
int get_neg(int n) {
    int k = abs(n) * 2;
    return n < 0 ? k + 1 : k;
}
int get_pos(int n) {
    int k = abs(n) * 2;
    return n < 0 ? k : k + 1;
}
const int mxn = 2e5 + 5;
vector<int> adj[mxn], adj_rev[mxn];
int32_t main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> m >> n;
    for (int i = 0; i < m; i++) {
        char c, d;
        int u, v;
        cin >> c >> u >> d >> v;
        if (c == '-') u = -u;
        if (d == '-') v = -v;
        adj[get_neg(u)].push_back(get_pos(v));
        adj_rev[get_pos(v)].push_back(get_neg(u));
        adj[get_neg(v)].push_back(get_pos(u));
        adj_rev[get_pos(u)].push_back(get_neg(v));
    }
    vector<int> vis(n * 2 + 5), topo;
    function<void(int)> dfs = [&](int u) {
        vis[u] = 1;
        for (int v : adj[u]) if (!vis[v]) dfs(v);
        topo.push_back(u);
    };
    for (int i = 2; i <= n * 2 + 1; i++) {
        if (!vis[i]) dfs(i);
    }
    reverse(all(topo));
    int cnt = 1;
    fill(all(vis), 0LL);
    vector<int> val(n * 2 + 5);
    function<void(int)> dfs2 = [&](int u) {
        vis[u] = 1;
        val[u] = cnt;
        for (int v : adj_rev[u]) if (!vis[v]) dfs2(v);
    };
    for (int i : topo) {
        if (!vis[i]) {
            dfs2(i);
            cnt++;
        }
    }
    for (int i = 1; i <= n; i++) {
        if (val[i * 2] == val[i * 2 + 1]) {
            cout << "IMPOSSIBLE";
            return 0;
        }
    }
    for (int i = 1; i <= n; i++) {
        if (val[i << 1 | 1] > val[i << 1]) cout << '+' << ' '
            ;
        else cout << '-' << ' ';
    }
    return 0;
}
```

4.2 articulation-point

```
// 0-indexed
struct ArticulationPoint {
```

```
 int n, dfs_cnt;
 vector<int> dfs_order, check;
 vector<vector<int>> adj;
 ArticulationPoint(int n) :
  n(n), adj(n),
  dfs_order(n), check(n),
  dfs_cnt(0) {}
 void AddEdge(int a, int b) {
  adj[a].push_back(b);
  adj[b].push_back(a);
 }
 int DFS(int cur, bool IsRoot = 0) {
  int ret = dfs_order[cur] = ++dfs_cnt, cnt = 0;
  for (const auto& nxt : adj[cur]) {
   if (!dfs_order[nxt]) {
    const int t = DFS(nxt);
    if (t >= dfs_order[cur]) check[cur] = 1;
    ret = min(ret, t), cnt++;
   }
   else ret = min(ret, dfs_order[nxt]);
  }
  if (IsRoot) check[cur] = cnt > 1;
  return ret;
 }
 void GetCheck() {
  for (int i = 0; i < n; i++)
   if (!dfs_order[i]) DFS(i, 1);
 }
};
// ArticulationPoint ap(n);
// ap.AddEdge()
// ap.GetCheck();
```

4.3 bridge

```
// 0-indexed
struct Bridge { // must be no multi-edge
 int n, dfs_cnt;
 vector<int> dfs_order;
 vector<pair<int, int>> bridge;
 vector<vector<int>> adj;
 Bridge(int n) :
  n(n), adj(n),
  dfs_order(n),
  dfs_cnt(0) {}
 void AddEdge(int a, int b) {
  adj[a].push_back(b);
  adj[b].push_back(a);
 }
 int DFS(int cur, int prv = -1) {
  int ret = dfs_order[cur] = ++dfs_cnt;
  for (const auto& nxt : adj[cur]) {
   if (nxt == prv) continue;
   if (!dfs_order[nxt]) {
    const int t = DFS(nxt, cur);
    if (t > dfs_order[cur]) bridge.push_back({ cur, nxt });
    ret = min(ret, t);
   }
   else ret = min(ret, dfs_order[nxt]);
  }
  return ret;
 }
 void GetBridge() {
  for (int i = 0; i < n; i++)
   if (!dfs_order[i]) DFS(i);
  for (auto& [a, b] : bridge) if (a > b) swap(a, b);
  sort(bridge.begin(), bridge.end());
 }
};
// Bridge b(n);
// b.AddEdge()
// b.GetBridge();
```

4.4 flow

```
template <typename T>
class flow_graph {
public:
    static constexpr T eps = (T)1e-9;

    struct edge {
        int to;
        T c;
        T f;
        int rev;
    };

    vector<vector<edge>> g;
    vector<int> ptr;
    vector<int> d;
    vector<int> q;
    vector<int> cnt_on_layer;
    vector<int> prev_edge;
    bool can_reach_sink;

    int n;
    int st, fin;
    T flow;

    flow_graph(int _n, int _st, int _fin) : n(_n), st(_st),
        fin(_fin) {
        assert(0 <= st && st < n && 0 <= fin && fin < n&& st
            != fin);
        g.resize(n);
        ptr.resize(n);
        d.resize(n);
        q.resize(n);
        cnt_on_layer.resize(n + 1);
        prev_edge.resize(n);
        flow = 0;
    }

    void clear_flow() {
        for (int i = 0; i < n; i++) {
            for (edge& e : g[i]) {
                e.f = 0;
            }
        }
        flow = 0;
    }

    void add(int from, int to, T forward_cap, T backward_cap
        = 0) {
        assert(0 <= from && from < n && 0 <= to && to < n);
        int from_size = g[from].size();
        int to_size = g[to].size();
        g[from].push_back({ to, forward_cap, 0, to_size });
        g[to].push_back({ from, backward_cap, 0, from_size })
            ;
    }

    bool expath() {
        fill(d.begin(), d.end(), n);
        q[0] = fin;
        d[fin] = 0;
        fill(cnt_on_layer.begin(), cnt_on_layer.end(), 0);
        cnt_on_layer[n] = n - 1;
        cnt_on_layer[0] = 1;
        int beg = 0, end = 1;
        while (beg < end) {
            int i = q[beg++];
            for (const edge& e : g[i]) {
                const edge& back = g[e.to][e.rev];
                if (back.c - back.f > eps && d[e.to] == n) {
                    cnt_on_layer[d[e.to]]--;
                    d[e.to] = d[i] + 1;
                    cnt_on_layer[d[e.to]]++;
                    q[end++] = e.to;
```

```
            }
        }
    }
    return (d[st] != n);
}

T augment(int& v) {
    T cur = numeric_limits<T>::max();
    int i = fin;
    while (i != st) {
        const edge& e = g[i][prev_edge[i]];
        const edge& back = g[e.to][e.rev];
        cur = min(cur, back.c - back.f);
        i = e.to;
    }
    i = fin;
    while (i != st) {
        edge& e = g[i][prev_edge[i]];
        edge& back = g[e.to][e.rev];
        back.f += cur;
        e.f -= cur;
        i = e.to;
        if (back.c - back.f <= eps) {
            v = i;
        }
    }
    return cur;
}

int retreat(int v) {
    int new_dist = n - 1;
    for (const edge& e : g[v]) {
        if (e.c - e.f > eps && d[e.to] < new_dist) {
            new_dist = d[e.to];
        }
    }
    cnt_on_layer[d[v]]--;
    if (cnt_on_layer[d[v]] == 0) {
        if (new_dist + 1 > d[v]) {
            can_reach_sink = false;
        }
    }
    d[v] = new_dist + 1;
    cnt_on_layer[d[v]]++;
    if (v != st) {
        v = g[v][prev_edge[v]].to;
    }
    return v;
}

T max_flow() {
    can_reach_sink = true;
    for (int i = 0; i < n; i++) {
        ptr[i] = (int)g[i].size() - 1;
    }
    if (expath()) {
        int v = st;
        while (d[st] < n) {
            while (ptr[v] >= 0) {
                const edge& e = g[v][ptr[v]];
                if (e.c - e.f > eps && d[e.to] == d[v] -
                    1) {
                    prev_edge[e.to] = e.rev;
                    v = e.to;
                    if (v == fin) {
                        flow += augment(v);
                    }
                    break;
                }
                ptr[v]--;
            }
            if (ptr[v] < 0) {
                ptr[v] = (int)g[v].size() - 1;
                v = retreat(v);
                if (!can_reach_sink) {
                    break;
                }
            }
        }
    }
    return flow;
}

vector<bool> min_cut() {
    max_flow();
    assert(!expath());
    vector<bool> ret(n);
    for (int i = 0; i < n; i++) {
        ret[i] = (d[i] != n);
    }
    return ret;
}
/*
      must call min_cut() when need find min_cut()
      expath go from fin
      define d[u] is distance from fin , if d[u] = n -> u
            can reach from fin
      if an edge have u->v and d[u] == n && d[v] != n ->
           (u, v) is min_cut
*/
};
```

4.5 hopcroft-karp

```
// minimum vertex cover = maximum matching
// maximum independent set = number_of_vertex - maximum
    matching
// 0 - indexed
vector<pair<int, int>> maxMatch(int L, int R, const vector<
    pair<int, int>>& edges) {
 vector<vector<int>> adj = vector<vector<int>>(L);
 vector<int> nxt(L, -1), prv(R, -1), lev, ptr;
 for (int i = 0; i < edges.size(); i++) adj.at(edges[i].
     first).push_back(edges[i].second);
 while (true) {
  lev = ptr = vector<int>(L); int max_lev = 0;
  queue<int> q; for (int i = 0; i < L; i++) if (nxt[i] ==
      -1) lev[i] = 1, q.push(i);
  while (q.size()) {
   int x = q.front(); q.pop();
   for (int y : adj[x]) {
    int z = prv[y];
    if (z == -1) max_lev = lev[x];
    else if (!lev[z]) lev[z] = lev[x] + 1, q.push(z);
   }
   if (max_lev) break;
  }
  if (!max_lev) break;
  for (int i = 0; i < L; i++) if (lev[i] > max_lev) lev[i] =
      0;
  auto dfs = [&](auto self, int x) -> bool {
   for (; ptr[x] < adj[x].size(); ++ptr[x]) {
    int y = adj[x][ptr[x]], z = prv[y];
    if (z == -1 || (lev[z] == lev[x] + 1 && self(self, z)))
     return nxt[x] = y, prv[y] = x, ptr[x] = adj[x].size(),
         1;
   }
   return 0;
  };
  for (int i = 0; i < L; i++) if (nxt[i] == -1) dfs(dfs, i);
 }
 vector<pair<int, int>> ans; for (int i = 0; i < L; i++) if
     (nxt[i] != -1) ans.push_back({ i,nxt[i] });
 return ans;
}
```

4.6 min-cost-max-flow

```cpp
#include <bits/extc++.h>
template <typename T, typename C>
class MCMF {
public:
    static constexpr T eps = (T)1e-9;
    struct edge {
        int from, to;
        T c, f;
        C cost;
    };
    int n;
    vector<vector<int>> g;
    vector<edge> edges;
    vector<C> d, pot;
    __gnu_pbds::priority_queue<pair<C, int>> q;
    vector<typename decltype(q)::point_iterator> its;
    vector<int> pe;
    const C INF_C = numeric_limits<C>::max() / 2;
    explicit MCMF(int n_) : n(n_), g(n), d(n), pot(n, 0), its
        (n), pe(n) {}
    int add(int from, int to, T forward_cap, T backward_cap,
        C edge_cost) {
        assert(0 <= from && from < n && 0 <= to && to < n);
        assert(forward_cap >= 0 && backward_cap >= 0);
        int id = static_cast<int>(edges.size());
        g[from].push_back(id);
        edges.push_back({ from, to, forward_cap, 0, edge_cost
            });
        g[to].push_back(id + 1);
        edges.push_back({ to, from, backward_cap, 0, -
            edge_cost });
        return id;
    }

    void expath(int st) {
        fill(d.begin(), d.end(), INF_C);
        q.clear();
        fill(its.begin(), its.end(), q.end());
        its[st] = q.push({ pot[st], st });
        d[st] = 0;
        while (!q.empty()) {
            int i = q.top().second;
            q.pop();
            its[i] = q.end();
            for (int id : g[i]) {
                const edge& e = edges[id];
                int j = e.to;
                if (e.c - e.f > eps && d[i] + e.cost < d[j]) {
                    d[j] = d[i] + e.cost;
                    pe[j] = id;
                    if (its[j] == q.end()) {
                        its[j] = q.push({ pot[j] - d[j], j });
                    }
                    else {
                        q.modify(its[j], { pot[j] - d[j], j });
                    }
                }
            }
        }
        swap(d, pot);
    }
    pair<T, C> max_flow_min_cost(int st, int fin) {
        T flow = 0;
        C cost = 0;
        bool ok = true;
        for (auto& e : edges) {
            if (e.c - e.f > eps && e.cost + pot[e.from] - pot
                [e.to] < 0) {
                ok = false;
                break;
            }
        }
        if (ok) {
            expath(st);
        }
        else {
            vector<int> deg(n, 0);
            for (int i = 0; i < n; i++) {
                for (int eid : g[i]) {
                    auto& e = edges[eid];
                    if (e.c - e.f > eps) {
                        deg[e.to] += 1;
                    }
                }
            }
            vector<int> que;
            for (int i = 0; i < n; i++) {
                if (deg[i] == 0) {
                    que.push_back(i);
                }
            }
            for (int b = 0; b < (int)que.size(); b++) {
                for (int eid : g[que[b]]) {
                    auto& e = edges[eid];
                    if (e.c - e.f > eps) {
                        deg[e.to] -= 1;
                        if (deg[e.to] == 0) {
                            que.push_back(e.to);
                        }
                    }
                }
            }
            fill(pot.begin(), pot.end(), INF_C);
            pot[st] = 0;
            if (static_cast<int>(que.size()) == n) {
                for (int v : que) {
                    if (pot[v] < INF_C) {
                        for (int eid : g[v]) {
                            auto& e = edges[eid];
                            if (e.c - e.f > eps) {
                                if (pot[v] + e.cost < pot[e.to
                                    ]) {
                                    pot[e.to] = pot[v] + e.cost;
                                    pe[e.to] = eid;
                                }
                            }
                        }
                    }
                }
            }
            else {
                que.assign(1, st);
                vector<bool> in_queue(n, false);
                in_queue[st] = true;
                for (int b = 0; b < (int)que.size(); b++) {
                    int i = que[b];
                    in_queue[i] = false;
                    for (int id : g[i]) {
                        const edge& e = edges[id];
                        if (e.c - e.f > eps && pot[i] + e.cost
                            < pot[e.to]) {
                            pot[e.to] = pot[i] + e.cost;
                            pe[e.to] = id;
                            if (!in_queue[e.to]) {
                                que.push_back(e.to);
                                in_queue[e.to] = true;
                            }
                        }
                    }
                }
            }
        }
        while (pot[fin] < INF_C) {
            T push = numeric_limits<T>::max();
            int v = fin;
            while (v != st) {
                const edge& e = edges[pe[v]];
                push = min(push, e.c - e.f);
                v = e.from;
            }
```

```
            v = fin;
            while (v != st) {
                edge& e = edges[pe[v]];
                e.f += push;
                edge& back = edges[pe[v] ^ 1];
                back.f -= push;
                v = e.from;
            }
            flow += push;
            cost += push * pot[fin];
            expath(st);
        }
        return { flow, cost };
    }
};
```

4.7 minimum-vertex-cover

```
vector<vector<int>> minimumVertexCover(int L, int R, const
    vector<pair<int, int>>& edges) {
 vector<vector<int>> adjl(L), adjr(R);
 vector<int> in_maxMatch(L);
 auto p = maxMatch(L, R, edges);
 for (auto x : edges) {
  if (find(all(p), x) != p.end()) {
   in_maxMatch[x.first] = 1;
   adjr[x.second].push_back(x.first);
  }
  else {
   adjl[x.first].push_back(x.second);
  }
 }
 vector<int> visl(L), visr(R);
 function<void(int, int)> dfs = [&](int u, int p) {
  if (p) {
   visr[u] = 1;
   for (int v : adjr[u]) {
    if (!visl[v]) dfs(v, 0);
   }
  }
  else {
   visl[u] = 1;
   for (int v : adjl[u]) {
    if (!visr[v]) dfs(v, 1);
   }
  }
 };
 for (int i = 0; i < L; i++) {
  if (!in_maxMatch[i] && !visl[i]) dfs(i, 0);
 }
 vector<vector<int>> ans(2);
 for (int i = 0; i < L; i++) if (!visl[i]) ans[0].push_back(
     i);
 for (int i = 0; i < R; i++) if (visr[i]) ans[1].push_back(i
     );
 return ans;
}
```

4.8 spfa

```
const int INF = 1e18;
int main()
{
 int n, m, s, t;
 vector<vector<pair<int, int>>> adj(n);
 int inque_dist_sum = 0;
 vector<int> dist(n, INF);
 vector<int> trace(n, -1);
 vector<bool> inqueue(n, false);
 deque<int> que;
 que.push_back(s);
 dist[s] = 0;
 inqueue[s] = true;
 while (que.size()) {
  auto i = que.front();
  que.pop_front();
  inque_dist_sum -= dist[i];
  inqueue[i] = false;
  while (que.size() && dist[que.front()] * que.size() >
      inque_dist_sum) que.push_back(que.front()), que.
      pop_front();
  for (auto j : adj[i]) if (dist[j.first] > dist[i] + j.
      second) {
   if (inqueue[j.first]) inque_dist_sum -= dist[j.first];
   dist[j.first] = dist[i] + j.second;
   trace[j.first] = i;
   inque_dist_sum += dist[j.first];
   if (!inqueue[j.first]) {
    inqueue[j.first] = true;
    if (dist[j.first] * que.size() > inque_dist_sum) que.
        push_back(j.first);
    else que.push_front(j.first);
   }
  }
 }
}
```

5 math

5.1 chinese-remainer

```
#include "extended_euclid.h"
#define int long long
int chinese(int a, int m, int b, int n) {
    int32_t x, y;
    euclid(m, n, x, y);
    int ret = a * (y + m) % m * n + b * (x + n) % n * m;
    if (ret >= m * n) ret -= m * n;
    return ret;
}
int chinese_common(int a, int m, int b, int n) {
    int d = __gcd(m, n);
    b = ((b - a) % n + n) % n;
    if (b % d != 0) return -1;
    return d * chinese(0, m/d, b/d, n/d) + a;
}
```

5.2 count-prime

```
struct _count_primes_struct_t_ {
    vector<int> primes, mnprimes;
    int ans, y;
    vector<pair<pair<int, int>, char>> queries;
    int count_primes(int n) {
        // this y is actually n/y
        // also no logarithms, welcome to reality, this y is
            the best for n=10^12 or n=10^13
        y = pow(n, 0.64);
        if (n < 100) y = n;
        // linear sieve
        primes.clear();
        mnprimes.assign(y + 1, -1);
        ans = 0;
        for (int i = 2; i <= y; ++i) {
            if (mnprimes[i] == -1) {
                mnprimes[i] = primes.size();
                primes.push_back(i);
            }
            for (int k = 0; k < primes.size(); ++k) {
                int j = primes[k];
                if (i * j > y) break;
                mnprimes[i * j] = k;
                if (i % j == 0) break;
            }
        }
```

```
    if (n < 100) return primes.size();
    int s = n / y;
    for (int p : primes) {
        if (p > s) break;
        ans++;
    }
    // pi(n / y)
    int ssz = ans;
    // F with two pointers
    int ptr = primes.size() - 1;
    for (int i = ssz; i < primes.size(); ++i) {
        while (ptr >= i && (int)primes[i] * primes[ptr] >
              n)
            --ptr;
        if (ptr < i) break;
        ans -= ptr - i + 1;
    }
    // phi, store all queries
    phi(n, ssz - 1);
    sort(queries.begin(), queries.end());
    int ind = 2;
    int sz = primes.size();
    // the order in fenwick will be reversed, because
         prefix sum in a fenwick is just one query
    fenwick fw(sz);
    for (auto [na, sign] : queries) {
        auto [n, a] = na;
        while (ind <= n)
            fw.add(sz - 1 - mnprimes[ind++], 1);
        ans += (fw.ask(sz - a - 2) + 1) * sign;
    }
    queries.clear();
    return ans - 1;
}
void phi(int n, int a, int sign = 1) {
    if (n == 0) return;
    if (a == -1) {
        ans += n * sign;
        return;
    }
    if (n <= y) {
        queries.emplace_back(make_pair(n, a), sign);
        return;
    }
    phi(n, a - 1, sign);
    phi(n / primes[a], a - 1, -sign);
}
struct fenwick {
    vector<int> tree;
    int n;

        fenwick(int n = 0) : n(n) {
            tree.assign(n, 0);
        }
        void add(int i, int k) {
            for (; i < n; i = (i | (i + 1)))
                tree[i] += k;
        }
        int ask(int r) {
            int res = 0;
            for (; r >= 0; r = (r & (r + 1)) - 1)
                res += tree[r];
            return res;
        }
    };
} _count_primes_struct_;
```

5.3 eulerphi

```
int eulerPhi(int n) { // = n (1-1/p1) ... (1-1/pn)
    if (n == 0) return 0;
    int ans = n;
    for (int x = 2; x*x <= n; ++x) {
        if (n % x == 0) {
            ans -= ans / x;
            while (n % x == 0) n /= x;
        }
    }
    if (n > 1) ans -= ans / n;
    return ans;
}
// Phi ham euler dung sang
vector<int> buildEulerPhi(int N) {
    vector<int> phi(N+1);
    for (int i = 1; i <= N; i++)
        phi[i] = i;
    for (int i = 2; i <= N; i++)
        if (phi[i] == i)
            for (int j = i; j <= N; j += i)
                phi[j] -= phi[j]/i;
    return phi;
}
```

5.4 extended-euclid

```
// Dung extended euclid de tim nghiem cua phuong trinh
// a*x + b*y = gcd(a, b)
int euclid(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1 != 0) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    if (a1 < 0) {
        a1 = -a1;
        x = -x;
        y = -y;
    }
    return a1;
}

// Tinh nghich dao su dung extended euclid
// Tra ve -1 neu khong ton tai nghich dao
int inverse_euclid(int a, int mod) {
    int x, y;
    int g = euclid(a, mod, x, y);
    if(g == 1) {
        x = (x % mod + mod) % mod;
        return x;
    }
    else {
        // No solution
        return -1;
    }
}

// Giai phuong trinh (ax + by = c)
// Neu ton tai tao ra mot ho nghiem, do la (x + dx * k) va (
     y + dy * k)
bool solve_extended_euclid(int a, int b, int c, int &x, int&
      dx, int &y, int &dy) {
    assert(a != 0 || b != 0);
    int g = euclid(a, b, x, y);
    if (c % g != 0) return false;
    x *= c / g;
    y *= c / g;
    dx = b / g;
    dy = -a / g;
    return true;
}
```

5.5 factorization

```cpp
#include <bits/stdc++.h>
using namespace std;
auto seed = chrono::high_resolution_clock::now().
    time_since_epoch().count();
mt19937 mt(seed);

long long pollardRho(long long n) {
    if (n % 2 == 0)
        return 2;
    long long x, y, g = 1, a;
    x = mt() % n + 1;
    y = x;
    a = mt() % n + 1;
    while (g == 1 || g == n) {
        if (g == n)
        {
            x = mt() % n + 1;
            y = x;
            a = mt() % n + 1;
        }
        x = (((__int128)x * x) + a) % n;
        y = (((__int128)y * y) + a) % n;
        y = (((__int128)y * y) + a) % n;
        g = gcd(abs(x - y), n);
    }
    return g;
}
inline uint64_t mod_mult64(uint64_t a, uint64_t b, uint64_t
    m)
{
    return __int128_t(a) * b % m;
}
uint64_t mod_pow64(uint64_t a, uint64_t b, uint64_t m)
{
    uint64_t ret = (m > 1);
    for (;;)
    {
        if (b & 1) ret = mod_mult64(ret, a, m);
        if (!(b >>= 1)) return ret;
        a = mod_mult64(a, a, m);
    }
}

// Works for all primes p < 2^64
bool is_prime(uint64_t n)
{
    if (n <= 3) return (n >= 2);
    static const uint64_t small[] =
    {
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
            47, 53, 59, 61, 67,
        71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127,
            131, 137, 139,
        149, 151, 157, 163, 167, 173, 179, 181, 191, 193,
            197, 199,
    };
    for (size_t i = 0; i < sizeof(small) / sizeof(uint64_t);
        ++i)
    {
        if (n % small[i] == 0) return n == small[i];
    }

    // Makes use of the known bounds for Miller-Rabin
        pseudoprimes.
    static const uint64_t millerrabin[] =
    {
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
    };
    static const uint64_t A014233[] = // From OEIS.
    {
        2047LL, 1373653LL, 25326001LL, 3215031751LL,
            2152302898747LL,
        3474749660383LL, 341550071728321LL, 341550071728321LL
            ,
        3825123056546413051LL, 3825123056546413051LL,
            3825123056546413051LL, 0,
    };
    uint64_t s = n - 1, r = 0;
    while (s % 2 == 0)
    {
        s /= 2;
        r++;
    }
    for (size_t i = 0, j; i < sizeof(millerrabin) / sizeof(
        uint64_t); i++)
    {
        uint64_t md = mod_pow64(millerrabin[i], s, n);
        if (md != 1)
        {
            for (j = 1; j < r; j++)
            {
                if (md == n - 1) break;
                md = mod_mult64(md, md, n);
            }
            if (md != n - 1) return false;
        }
        if (n < A014233[i]) return true;
    }
    return true;
}
vector<pair<long long, int>> factorize(long long n) {
    assert(n > 1);
    vector<pair<long long, int>> res;
    for (long long i = 2; i * i * i <= n; i++) {
        if (n % i == 0) {
            res.emplace_back(i, 0);
            while (n % i == 0) {
                res.back().second++;
                n /= i;
            }
        }
    }
    if (n == 1) return res;
    if (is_prime(n)) {
        res.emplace_back(n, 1);
        return res;
    }
    long long k = pollardRho(n);
    if (k > n / k) k = n / k;
    res.emplace_back(k, 1);
    if (k * k == n) res.back().second++;
    else res.emplace_back(n / k, 1);
    sort(res.begin(), res.end());
    return res;
}
```

5.6 fft

```cpp
#define sqr(x) ((x) * (x))

struct Cmplx {
    double x, y;
    Cmplx(double x = 0, double y = 0) {this->x = x; this->y =
        y;}
    double real() {return x;}
    double imag() {return y;}
};
Cmplx operator + (Cmplx a, Cmplx b) {return Cmplx(a.x + b.x,
    a.y + b.y); }
Cmplx operator - (Cmplx a, Cmplx b) { return Cmplx(a.x - b.x
    , a.y - b.y); }
Cmplx operator * (Cmplx a, Cmplx b) { return Cmplx(a.x*b.x -
    a.y*b.y, a.x*b.y + a.y*b.x); }
Cmplx operator * (Cmplx a, double b) { return Cmplx(a.x*b, a
    .y*b); }
Cmplx operator / (Cmplx a, double b) { return Cmplx(a.x/b, a
    .y/b); }
Cmplx operator / (Cmplx a, Cmplx b) {
```

```
    return Cmplx(a.x*b.x + a.y*b.y, a.y*b.x - a.x*b.y) / (sqr
        (b.x) + sqr(b.y));
}
Cmplx conj(Cmplx a) {return Cmplx(a.x, -a.y);}

using cd = Cmplx;
//using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd>& a, int n, bool invert) {
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while(j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if(i < j) swap(a[i], a[j]);
    }

    for(int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len;
        if(invert) ang = -ang;
        cd wlen(cos(ang), sin(ang));
        for(int i = 0; i < n; i += len) {
            cd w(1);
            for(int j = 0; j < len/2; j++) {
                cd u = a[i + j];
                cd v = a[i + j + len/2] * w;
                a[i + j] = u + v;
                a[i + j + len/2] = u - v;
                w = w * wlen;
            }
        }
    }
    if(invert) {
        for(cd & x : a) x = x / (double)n;
    }
}

vector<cd> f1;
vector<cd> f2;
vector<int64_t> multiply(const vector<int>& a, const vector<
     int>& b) {
    int need = a.size() + b.size() - 1;
    int sz = 1;
    while (sz < need) sz <<= 1;
    if(sz > f1.size()) {
        f1.resize(sz);
        f2.resize(sz);
    }
    for(int i = 0; i < sz; i++) {
        double x = (i < a.size() ? a[i] : 0);
        double y = (i < b.size() ? b[i] : 0);
        f1[i] = cd(x, y);
    }
    fft(f1, sz, false);
    for(int i = 0; i < sz; i++) {
        int j = (sz - i) & (sz - 1);
        cd x = (f1[i] + conj(f1[j])) / cd(2,0);
        cd y = (f1[i] - conj(f1[j])) / cd(0,2);
        f2[i] = x*y;
    }
    fft(f2, sz, true);
    vector<int64_t> res(need);
    for (int i = 0; i < need; i++) res[i] = llround(f2[i].
         real());
    return res;
}
```

5.7 fraction

```
int cmp(int a, int b) { return (a == b) ? 0 : ((a < b) ? -1
     : 1); }
struct Fraction {
    int a, b;
    Fraction() {
        a = 0; b = 1;
    }
    Fraction(int _a, int _b) {
        a = _a; b = _b; norm();
    }
    Fraction(int x) {
        a = x; b = 1;
    }
    Fraction operator + (const Fraction& other) const {
        return Fraction(a * other.b + b * other.a, b * other.
            b);
    }
    Fraction operator - (const Fraction& other) const {
        return Fraction(a * other.b - b * other.a, b * other.
            b);
    }
    Fraction operator * (const Fraction& other) const {
        return Fraction(a * other.a, b * other.b);
    }
    Fraction operator / (const Fraction& other) const {
        assert(other.a != 0);
        return Fraction(a * other.b, b * other.a);
    }
    int cmp(Fraction other) const {
        return ::cmp(a * other.b, b * other.a);
    }

#define Comp(x) bool operator x (Fraction q) const { return
     cmp(q) x 0; }
    Comp(>) Comp(<) Comp(==) Comp(>=) Comp(<=) Comp(!=)
#undef Comp
    void norm() {
        if (b < 0) {
            a = -a; b = -b;
        }
        if (a == 0) b = 1;
        else {
            int g = __gcd(llabs(a), llabs(b));
            a /= g; b /= g;
        }
    }
};
istream& operator >> (istream& cin, Fraction& p) {
    cin >> p.a;
    p.b = 1;
    return cin;
}
ostream& operator << (ostream& cout, Fraction& p) {
    cout << p.a << '/' << p.b;
    return cout;
}
```

5.8 gauss

```
struct matrix {
    vector<vector<int>> a;
    int n, m;
    matrix(int n, int m) {
        this->n = n; this->m = m;
        a = vector<vector<int>>(n, vector<int>(m));
    }
};
int _gcd(int a, int b) {
    return b == 0 ? a : _gcd(b, a%b);
}
// rut gon cac so trong a
void norm_array(vector<int>& a) {
    int g = 0;
    for(int i : a)
        g = _gcd(g, abs(i));
    for(int i : a)
```

```
        if(i != 0) {
            if(i < 0)
                g *= -1;
            break;
        }
    for(int &i : a) i /= g;
}
void gauss_elimination(matrix& mat) {
    int n = mat.n, m = mat.m;
    for(int i = 0; i < n-1; i++) {
        // tim hang co a[i][i] != 0
        for(int j = i; j < n; j++) {
            if(mat.a[j][i] != 0) {
                swap(mat.a[j], mat.a[i]);
                break;
            }
        }
        // rut gon cac hang duoi
        for(int j = i+1; j < n; j++) {
            for(int k = m-1; k >= i; k--) {
                mat.a[j][k] = mat.a[j][k] * mat.a[i][i] - mat.
                     a[i][k] * mat.a[j][i];
            }
            norm_array(mat.a[j]);
        }
    }
    for(int i = 1; i < n; i++) {
        // rut gon cac hang tren
        for(int j = i-1; j >= 0; j--) {
            for(int k = m-1; k >= i; k--) {
                mat.a[j][k] = mat.a[j][k] * mat.a[i][i] - mat.
                     a[i][k] * mat.a[j][i];
            }
            for(int k = i-1; k >= 0; k--)
                mat.a[j][k] *= mat.a[i][i];
        }
    }
}
```

5.9 ntt

```
// MOD thay the:
// 1004535809 = 479 * 2^21 + 1
// root = 702606812, root_pw = 1 << 21
// 1012924417 = 483 * 2^21 + 1
// root = 673144645, root_pw = 1 << 21
// 998244353 = 119 * 2^23 + 1
// root = 15311432, root_pw = 1 << 23
int mod = 998244353; // // 1 + 7*17*2^23
int inv_2 = (mod+1) / 2;
int root = 15311432;
int root_pw = 1 << 23;
int inv_root = inverse(root, mod);
using ll = long long;
void fft(vector<ll> &a, int n, bool invert) {
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while(j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll wlen = invert ? inv_root : root;
        for (int i = len; i < root_pw; i <<= 1) wlen = wlen *
             wlen % mod;
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j];
                int v = a[i+j+len/2] * w % mod;
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v +
                     mod;
                w = w * wlen % mod;
            }
        }
    }
    if (invert) {
        int inv_n = inverse(n, mod);
        for(int i = 0; i < n; i++) a[i] = a[i] * inv_n % mod;
    }
}

vector<int> multiply(vector<int> a, vector<int> b) {
    vector<ll> fa(a.begin(), a.end());
    vector<ll> fb(b.begin(), b.end());
    int n = 1;
    int sz = a.size() + b.size() - 1;
    while(n < sz) n <<= 1;
    fa.resize(n), fb.resize(n);
    fft(fa, n, false);
    fft(fb, n, false);
    for(int i = 0; i < n; i++) fa[i] = fa[i] * fb[i] % mod;
    fft(fa, n, true);
    vector<int> res(sz);
    for(int i = 0; i < sz; i++) res[i] = fa[i];
    return res;
}
```

5.10 primitive-root

```
#include <bits/stdc++.h>
using namespace std;
typedef complex<double> cd;

int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 1ll * a % p), --b;
        else
            a = int (a * 1ll * a % p), b >>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}
// root^(p-1) = 1
// 1012924417 = 3*7*23 * 2^21 + 1
// root = 673144645, inv_root = 915669194
// 998244353 = 7*17 * 2^23 + 1
// root = 15311432, inv_root = 469870224
void test_primitive_root() {
    int mod = 998244353;
    int root = powmod(generator(mod), 7*17, mod);
    int inv_root = powmod(root, mod-2, mod);
    cout << root << " " << inv_root << "\n";
```

```
}
```

5.11 sieve-range

```
int segmentedSieveNoPreGen(long long L, long long R) {
    vector<bool> isPrime(R - L + 1, true);
    long long lim = sqrt(R);
    for (long long i = 2; i <= lim; ++i)
        for (long long j = max(i * i, (L + i - 1) / i * i); j
             <= R; j += i)
            isPrime[j - L] = false;
    if (L == 1)
        isPrime[0] = false;
    int ans = R - L + 1;
    for (int i : isPrime) ans -= !i;
    return ans;
}
```

5.12 xor-transform

```
const int mod = 998244353;

void fwht(int* a, int len, bool invert) {
    // assert(len > 0 && (len & (len-1)) == 0);
    for (int h = 1; h < len; h <<= 1) {
        for (int i = 0; i < len; i += (h << 1)) {
            for (int j = i; j < i + h; j++) {
                int x = a[j];
                int y = a[j + h];
                a[j] = x + y;
                a[j + h] = x - y;
                if(a[j] >= mod) a[j] -= mod;
                if(a[j+h] < 0) a[j+h] += mod;
            }
        }
    }
    if(invert) {
        int inv_len = inverse(len);
        for(int i = 0; i < len; i++) {
            a[i] = 1LL * a[i] * inv_len % mod;
        }
    }
}
```

6 misc

6.1 compare-function

```
class Compare {
public:
    bool operator() (const string &a, const string &b) const
        {
        return a + b > b + a;
    }
};
```

6.2 hash-un-weighted-tree

```
auto seed = chrono::high_resolution_clock::now().
     time_since_epoch().count();
mt19937_64 mt(seed);
long long get_random(long long l, long long r) {
 assert(l <= r);
 return mt() % (r - l + 1) + l;
}
struct hash_unweighted_tree{
 // NOTE : undirected graph
 const int mod1 = 2002577573;
 const int mod2 = 2137321811;
 int MAX_DEPTH;
 vector<int> rnd;
 void init(int MAX_DEPTH) {// maybe number of vertex?
  this->MAX_DEPTH = MAX_DEPTH;
  rnd.resize(MAX_DEPTH);
  for (int i = 0; i < MAX_DEPTH; i++) rnd[i] = get_random(1,
       min(mod1, mod2) - 1);
 }
 vector<int> Centroid(const vector<vector<int>> &adj){ //
      get centroid, each tree have 1 or 2 centroid(s)
     int n = adj.size();
     vector<int> centroid;
     vector<int> sz(n);
     function<void (int, int)> dfs = [&](int u, int p) {
            sz[u] = 1;
            bool is_centroid = true;
            for (auto v : adj[u]) if (v != p) {
                    dfs(v, u);
                    sz[u] += sz[v];
                    if (sz[v] > n / 2) is_centroid = false;
            }
            if (n - sz[u] > n / 2) is_centroid = false;
            if (is_centroid) centroid.push_back(u);
     };
     dfs(0, -1);
     return centroid;
 }
 pair<int, int> get_hash(vector<vector<int>> adj, int root)
      {// root = -1 if using centroid as root
  int sz = adj.size();
  if (root == -1){
   auto c = Centroid(adj);
   if (c.size() == 2){
    adj[c[0]].erase(find(all(adj[c[0]]), c[1]));
    adj[c[1]].erase(find(all(adj[c[1]]), c[0]));
    adj[c[0]].push_back(sz);
    adj[c[1]].push_back(sz);
    adj.push_back({});
    adj[sz].push_back(c[0]);
    adj[sz].push_back(c[1]);
    root = sz;
    sz++;
   }
   else root = c[0];
  }
  vector<int> h1(sz, 1), h2(sz, 1), d(sz);
  function<void(int, int)> dfs = [&](int u, int p){
   int has = 0;
   vector<int> z1, z2;
   for (int v : adj[u]){
    if (v == p) continue;
    has = 1;
    dfs(v, u);
    ckmax(d[u], d[v] + 1);
    z1.push_back(h1[v]);
    z2.push_back(h2[v]);
   }
   if (!has) return;
   sort(all(z1));
   sort(all(z2));
   for (int i = 0; i < z1.size(); i++){
    h1[u] = (h1[u] * (rnd[d[u]] + z1[i])) % mod1;
    h2[u] = (h2[u] * (rnd[d[u]] + z2[i])) % mod2;
   }
  };
  dfs(root, -1);
  return {h1[root], h2[root]};
 }
};
```

6.3 int128

```
istream &operator>>(istream &is, __int128 &v) {
    string s;
    is>>s;
    v=0;
    for(auto &it:s) if(isdigit(it)) v=v*10+it-'0';
    if(s[0]=='-') v*=-1;
    return is;
}

ostream &operator<<(ostream &os,const __int128 &v) {
    if(v==0) return (os<<"0");
    __int128 num=v;
    if(v<0) os<<'-',num=-num;
    string s;
    for(;num>0;num/=10) s.push_back((char)(num%10)+'0');
    reverse(all(s));
    return (os<<s);
}
int32_t main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    __int128 x = 999999999;
    __int128 z = 999999999;
    x = x * x;
    x = x * x;
    x = x / z;
    x = x / z;
    x = x / z;
    cout << x;
    return 0;
}
```

6.4 main

```
// flags used (vscode only, add in task.json)
// "-DLOCAL"
// "-fsanitize=address,undefined"
// "-D_GLIBCXX_DEBUG"

// pragma define
// #pragma GCC optimize("O3")
// #pragma GCC target("avx2")
// #pragma GCC optimize("unroll-loops")

#include <bits/stdc++.h>
#define int long long
using namespace std;
#define MASK(i) (1LL << (i))
#define MASK_ALL(i) ((1LL << (i+1)) - 1)
#define sqr(x) ((x) * (x))
const int inf = 1.1e18;
const int mod = 998244353;
const double eps = 1e-6;

void precompute() {
    // precompute here
}

void test() {
    // write code here
}

int32_t main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
#ifdef LOCAL
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);
#endif
    precompute();
    int t;
    cin >> t;
    while(t--)
        test();
}
```

6.5 misc

```
manhattan -> chebyshev (a, b) = (x - y, x + y) , distance =
     max(|x1 - x2|, |y1 - y2|)
chebyshev -> manhattan (a, b) = ((x + y) / 2, (x - y) / 2),
     distance = |x1 - x2| + |y1 - y2|
```

6.6 random

```
auto seed = chrono::high_resolution_clock::now().
     time_since_epoch().count();
mt19937_64 mt(seed);
```

6.7 stresstest

```
int main() {
    cout << "Compiling..." << endl;
    system("g++ main.cpp -o main_1");
    system("g++ main_2.cpp -o main_2");
    for(int test = 1; test <= 2000; test++) {
        generate_test();
        cout << "Test " << test << ": ";
        if(system("cat input.txt | ./main_1 > output1.txt")
             || system("cat input.txt | ./main_2 > output2.
             txt")) {
            cout << "Runtime error\n";
            return 0;
        }
        if(system("diff output1.txt output2.txt") != 0) {
            cout << "Wrong!\n";
            return 0;
        }
        else {
            cout << "OK!\n";
        }
    }
}
```

7 string

7.1 aho-corasick

```
// suffix_link : nhay toi nut co do dai nho hon ma lon nhat
     va la suffix cua chuoi hien tai (vi la trie nen chuoi
     hien tai la 1 prefix cua mot chuoi da them vao)
// exit_link : la nut gan nhat sau khi nhay (> 0) lan va nut
      do la ket thuc cua 1 trong nhung chuoi da them vao
// trong ham get() tim nhung chuoi la suffix cua hien tai
     theo do dai giam dan
// khi bien doi sang euler tour, link [exit_link, p] neu
     exit_link = -1 thi cho no = 0 (vi dinh 0 khong la cua
     dinh nao het)
// co the nen dinh lai chi su dung nhung dinh co has = true,
      thay vi toan bo de tiet kiem bo nho
const int M = 26;
struct aho_corasick{
    struct node{
        int suffix_link = -1, exit_link = -1, nxt[M], tot =
             0;
        bool has = false;
        node() {fill(nxt, nxt + M, -1);}
    };
    vector<node> g = {node()};
    void insert_string(const string &s){
        int p = 0;
```

```
        for (char c: s){
            if (g[p].nxt[c - 'a'] == -1){
                g[p].nxt[c - 'a'] = g.size();
                g.emplace_back();
            }
            p = g[p].nxt[c - 'a'];
        }
        g[p].has = true;
        g[p].tot++;
    }
    void build_automaton(){
        for (deque<int> q = {0}; q.size(); q.pop_front()){
            int v = q.front(), suffix_link = g[v].suffix_link
                ;
            if (v) g[v].exit_link = g[suffix_link].has ?
                 suffix_link : g[suffix_link].exit_link;
            for (int i=0; i < M; i++){
                int &nxt = g[v].nxt[i], nxt_sf = v ? g[
                     suffix_link].nxt[i] : 0;
                if (nxt == -1) nxt = nxt_sf;
                else{
                    g[nxt].suffix_link = nxt_sf;
                    q.push_back(nxt);
                }
            }
        }
    }
    // check if have any string inserted : g[p].has || g[p].
         exit_link != -1
    int get(int p) {
        int res = 0;
        for (int v = g[p].has ? p : g[p].exit_link; v != -1;
             v = g[v].exit_link) {
            res += g[v].tot;
        }
        return res;
    }
    int feed(int u, int c) {
        return g[u].nxt[c];
    }
};
p[i] = ac.insert_string(s);
for (int i = 0; i < n; i++) {
    int par = ac.g[p[i]].exit_link;
    if (par == -1) adj[0].push_back(p[i]);
    else adj[par].push_back(p[i]);
}
// when get, pos = (ac.g[cur].has ? cur : ac.g[cur].
     exit_link);
```

7.2 duval-prefixes

```
template <typename T>
vector<int> duval_prefixes(int n, const T &s) {
  vector<int> z = z_function(n, s);
  vector<int> ans(n, 0);
  int i = 0, pos = 0;
  while (i < n) {
    int j = i, k = i;
    while (j < n) {
      j++;
      if (j > pos) {
        if (z[k] <= pos - k && s[z[k]] < s[k + z[k]]) {
          int shift = (pos - i) / (j - k) * (j - k);
          ans[pos] = ans[pos - shift] + shift;
        } else {
          ans[pos] = i;
        }
        pos++;
      }
      if (s[k] < s[j]) k = i; else
      if (!(s[j] < s[k])) k++; else
      else break;
    }
    while (i <= k) {
      i += j - k;
    }
  }
  return ans;
  // returns 0-indexed positions of the least cyclic shifts
       of all prefixes
}

template <typename T>
vector<int> duval_prefixes(const T &s) {
  return duval_prefixes((int) s.size(), s);
}
```

7.3 duval

```
template <typename T>
int duval(int n, const T &s) {
  assert(n >= 1);
  int i = 0, ans = 0;
  while (i < n) {
    ans = i;
    int j = i + 1, k = i;
    while (j < n + n && !(s[j % n] < s[k % n])) {
      if (s[k % n] < s[j % n]) {
        k = i;
      } else {
        k++;
      }
      j++;
    }
    while (i <= k) {
      i += j - k;
    }
  }
  return ans;
  // returns 0-indexed position of the least cyclic shift
}
template <typename T>
int duval(const T &s) {
  return duval((int) s.size(), s);
}
```

7.4 hash61

```
struct hash61 {
  static const uint64_t md = (1LL << 61) - 1;
  static uint64_t step;
  static vector<uint64_t> pw;

  uint64_t addmod(uint64_t a, uint64_t b) const {
    a += b;
    if (a >= md) a -= md;
    return a;
  }

  uint64_t submod(uint64_t a, uint64_t b) const {
    a += md - b;
    if (a >= md) a -= md;
    return a;
  }

  uint64_t mulmod(uint64_t a, uint64_t b) const {
    uint64_t l1 = (uint32_t) a, h1 = a >> 32, l2 = (uint32_t)
         b, h2 = b >> 32;
    uint64_t l = l1 * l2, m = l1 * h2 + l2 * h1, h = h1 * h2;
    uint64_t ret = (l & md) + (l >> 61) + (h << 3) + (m >>
         29) + (m << 35 >> 3) + 1;
    ret = (ret & md) + (ret >> 61);
    ret = (ret & md) + (ret >> 61);
    return ret - 1;
  }

  void ensure_pw(int sz) {
```

```
    int cur = (int) pw.size();
    if (cur < sz) {
      pw.resize(sz);
      for (int i = cur; i < sz; i++) {
        pw[i] = mulmod(pw[i - 1], step);
      }
    }
  }

  vector<uint64_t> pref;
  int n;
  hash61() {
  }
  template<typename T>
  hash61(const T& s) {
    n = (int) s.size();
    ensure_pw(n + 1);
    pref.resize(n + 1);
    pref[0] = 1;
    for (int i = 0; i < n; i++) {
      pref[i + 1] = addmod(mulmod(pref[i], step), s[i]);
    }
  }

  inline uint64_t operator()(const int from, const int to)
      const {
    assert(0 <= from && from <= to && to <= n - 1);
    return submod(pref[to + 1], mulmod(pref[from], pw[to -
        from + 1]));
  }
};
mt19937 rng(123123);
uint64_t hash61::step = (md >> 2) + rng() % (md >> 1);
vector<uint64_t> hash61::pw = vector<uint64_t>(1, 1);
```

7.5 hashing-2d

```
template<int mod> // define int long long?
struct hashing2D {
 vector<vector<int>> h;
 vector<int> px, py;
 int n, m;
 static const int basex = 256, basey = 652;
 // basex, basey is base, must difference
 hashing2D() {}
 hashing2D(vector<string>& s) {
  n = (int)s.size(), m = (int)s[0].size();
  h.assign(n + 1, vector<int>(m + 1, 0));
  px.assign(n + 1, 1);
  py.assign(m + 1, 1);
  for (int i = 0; i < n; i++) px[i + 1] = 1LL * px[i] *
      basex % mod;
  for (int i = 0; i < m; i++) py[i + 1] = 1LL * py[i] *
      basey % mod;
  for (int i = 0; i < n; i++) {
   for (int j = 0; j < m; j++) {
    h[i + 1][j + 1] = s[i][j];
   }
  }
  for (int i = 0; i <= n; i++) {
   for (int j = 0; j < m; j++) {
    h[i][j + 1] = (h[i][j + 1] + 1LL * h[i][j] * basey % mod)
        % mod;
   }
  }
  for (int i = 0; i < n; i++) {
   for (int j = 0; j <= m; j++) {
    h[i + 1][j] = (h[i + 1][j] + 1LL * h[i][j] * basex % mod)
        % mod;
   }
  }
 }
 int get_hash(int x1, int y1, int x2, int y2) { // 1-indexed
  assert(1 <= x1 && x1 <= x2 && x2 <= n);
  assert(1 <= y1 && y1 <= y2 && y2 <= m);
  x1--;
  y1--;
  int dx = x2 - x1, dy = y2 - y1;
  return (1LL * (h[x2][y2] - 1LL * h[x2][y1] * py[dy] % mod
      + mod) % mod -
  1LL * (h[x1][y2] - 1LL * h[x1][y1] * py[dy] % mod + mod) %
      mod * px[dx] % mod + mod) % mod;
 }
 int get_hash() {// whole vector2D
  return get_hash(1, 1, n, m);
 }
};
const int mod1 = 1035972859;
const int mod2 = 1704760909;
hashing2D<mod1>(v1).get_hash();
```

7.6 kmp

```
// prefix function: *length* of longest prefix which is also
    suffix:
//   pi[i] = max(k: s[0..k-1] == s[i-(k-1)..i]
//
// KMP {{{
template<typename Container>
std::vector<int> prefix_function(const Container& s) {
    int n = s.size();
    std::vector<int> pi(n);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

// Tested: https://oj.vnoi.info/problem/substr
// Return all positions (0-based) that pattern ‘pat‘ appears
    in ‘text‘
std::vector<int> kmp(const std::string& pat, const std::
    string& text) {
    auto pi = prefix_function(pat + '\0' + text);
    std::vector<int> res;
    for (size_t i = pi.size() - text.size(); i < pi.size();
        ++i) {
        if (pi[i] == (int)pat.size()) {
            res.push_back(i - 2 * pat.size());
        }
    }
    return res;
}

// Tested: https://oj.vnoi.info/problem/icpc22_mt_b
// Returns cnt[i] = # occurrences of prefix of length-i
// NOTE: cnt[0] = n+1 (0-length prefix appears n+1 times)
std::vector<int> prefix_occurrences(const string& s) {
    int n = s.size();
    auto pi = prefix_function(s);
    std::vector<int> res(n + 1);
    for (int i = 0; i < n; ++i) res[pi[i]]++;
    for (int i = n - 1; i > 0; --i) res[pi[i - 1]] += res[i];
    for (int i = 0; i <= n; ++i) res[i]++;
    return res;
}
// }}}
```

7.7 manacher

```
// Return <even_len, odd_len>
// - even_len[i] = length of longest palindrome centered at
    [i, i+1]
// - odd_len[i] = length of longest palindrome centered at i
```

```
vector<vector<int>> manacher(const string& s) {
    int n = s.size();
    vector<vector<int>> res = { vector<int>(n + 1, 0), vector
        <int>(n, 0) };

    for (int z = 0; z < 2; z++) {
        for (int i = 0, l = 0, r = 0; i < n; i++) {
            int t = r - i + !z;
            if (i < r) res[z][i] = min(t, res[z][l + t]);

            int l2 = i - res[z][i], r2 = i + res[z][i] - !z;
            while (l2 && r2 + 1 < n && s[l2 - 1] == s[r2 +
                1]) {
                ++res[z][i];
                --l2;
                ++r2;
            }
            if (r2 > r) {
                l = l2;
                r = r2;
            }
        }
        for (int i = 0; i < n; i++) {
            res[z][i] = 2 * res[z][i] + z;
        }
    }
    res[0].erase(res[0].begin(), res[0].begin() + 1);
    res[0].pop_back();
    return res;
}
```

7.8 suffix-array

```
template <typename T>
vector<int> suffix_array(int n, const T& s, int char_bound)
    {
    vector<int> a(n);
    if (n == 0) {
        return a;
    }
    if (char_bound != -1) {
        vector<int> aux(char_bound, 0);
        for (int i = 0; i < n; i++) {
            aux[s[i]]++;
        }
        int sum = 0;
        for (int i = 0; i < char_bound; i++) {
            int add = aux[i];
            aux[i] = sum;
            sum += add;
        }
        for (int i = 0; i < n; i++) {
            a[aux[s[i]]++] = i;
        }
    }
    else {
        iota(a.begin(), a.end(), 0);
        sort(a.begin(), a.end(), [&s](int i, int j) { return
            s[i] < s[j]; });
    }
    vector<int> sorted_by_second(n);
    vector<int> ptr_group(n);
    vector<int> new_group(n);
    vector<int> group(n);
    group[a[0]] = 0;
    for (int i = 1; i < n; i++) {
        group[a[i]] = group[a[i - 1]] + (!(s[a[i]] == s[a[i -
            1]]));
    }
    int cnt = group[a[n - 1]] + 1;
    int step = 1;
    while (cnt < n) {
        int at = 0;
        for (int i = n - step; i < n; i++) {
            sorted_by_second[at++] = i;
        }
        for (int i = 0; i < n; i++) {
            if (a[i] - step >= 0) {
                sorted_by_second[at++] = a[i] - step;
            }
        }
        for (int i = n - 1; i >= 0; i--) {
            ptr_group[group[a[i]]] = i;
        }
        for (int i = 0; i < n; i++) {
            int x = sorted_by_second[i];
            a[ptr_group[group[x]]++] = x;
        }
        new_group[a[0]] = 0;
        for (int i = 1; i < n; i++) {
            if (group[a[i]] != group[a[i - 1]]) {
                new_group[a[i]] = new_group[a[i - 1]] + 1;
            }
            else {
                int pre = (a[i - 1] + step >= n ? -1 : group[a
                    [i - 1] + step]);
                int cur = (a[i] + step >= n ? -1 : group[a[i]
                    + step]);
                new_group[a[i]] = new_group[a[i - 1]] + (pre
                    != cur);
            }
        }
        swap(group, new_group);
        cnt = group[a[n - 1]] + 1;
        step <<= 1;
    }
    return a;
}
// element in s has value in range [0, char_bound)
// if char_bound = -1 -> element in s have value in range (-
    inf, inf)
template <typename T>
vector<int> suffix_array(const T& s, int char_bound) {
    return suffix_array((int)s.size(), s, char_bound);
}

template <typename T>
vector<int> build_lcp(int n, const T& s, const vector<int>&
    sa) {
    assert((int)sa.size() == n);
    vector<int> pos(n);
    for (int i = 0; i < n; i++) {
        pos[sa[i]] = i;
    }
    vector<int> lcp(max(n - 1, 0));
    int k = 0;
    for (int i = 0; i < n; i++) {
        k = max(k - 1, 0);
        if (pos[i] == n - 1) {
            k = 0;
        }
        else {
            int j = sa[pos[i] + 1];
            while (i + k < n && j + k < n && s[i + k] == s[j
                + k]) {
                k++;
            }
            lcp[pos[i]] = k;
        }
    }
    return lcp;
}

template <typename T>
vector<int> build_lcp(const T& s, const vector<int>& sa) {
    return build_lcp((int)s.size(), s, sa);
}
```

7.9 z-function

```
// TESTED : https://judge.yosupo.jp/submission/355279
// z[i] = length of longest common prefix of s[0..N] and s[i
    ..N]
vector<int> zfunc(string s) {
    int n = s.length();
    vector<int> z(n);
    z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i
            ]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}
```